AI Plant Organizer - Stage 1

Task 1

Project Description

I love plants, and I have always wanted my house to be filled with a variety of beautiful plants that add a natural vibrance to my space. The only problem is that I tend not to be very good at caring for said plants, so they often end up dying or diseased, at which point the only hope for them is if my sister takes them and cares for them herself. As much as I love to see her and have her care for them, I would love to be able to do the same myself. I could spend a good chunk of my time researching how to care for the various plants I would like to have in my space, but it would be very helpful to have a tool that expedites that process, all while continuing to educate me so that I am not entirely reliant on the tool. This is the problem this project will attempt to support (not fix, just support).

The target users for this platform are those who are looking for a tool to support their ability to care for their plants. It is purely a software-oriented tool, and as such, it will not care for any of the plants itself, and is meant purely to support the abilities of said users by giving them the necessary resources to care for their plants themselves.

The AI agent has the role of supporting the user. As such, it will

- Have the ability to create timeslots in a calendar a user can rely on to determine when they should water their plants
- Adapt to a users' system calendar, provided by text or with an external calendar, to provide convenient times for the user to tend to their plants
- Set up notifications for the user that remind them when to water their plants and if they missed watering a plant some day
- Educate the user on plant information, answering questions about caring for a specific plant, what to do when certain signs of health problems arise, and where to place the plant
- Upon notification from the user of an unhealthy plant, the agent will use provided photos of the plant to determine the health of the plant, and will store these photos to keep track of the health of the plant (comparing current photos to previous ones to determine if the plant's health is improving or getting worse)
- Use provided information from the user about spaces they will put their plants to give more accurate recommendations for where certain plants should go
- Update a list of similar plants to the user recommended based off of plants that the user owns already or has indicated that they like/want (these recommendations will sit in a list separate from the plants the user owns)
- Search the web for plant information from an identified plant, should previous measures to find such information fail (see paragraph about the software system and AI models)

The primary challenge I encounter when it comes to caring for my plants is that I don't have a set routine or process I go through to ensure my plants are well cared for. Furthermore, I don't always know if they're in the right place or healthy, which is definitely cause for concern. I believe an AI agent is appropriate for this problem because its use to support users is effective in ensuring one can easily care for their collection of plants. Instead of one being anxious about whether they are properly caring for their plants, they can be assured through the tailored information from the

AI agent that their efforts are going towards their plants in an optimal way. Further, the agent adapting to a user's preferences offers a sense of personable treatment that every plant owner needs, since not every plant and space is alike. Of course, regardless of this fact, I believe it is necessary in any AI implementation to continually encourage users to critically analyze the problems they are facing, and as such, I expect the AI agent to encourage users to also learn from the information it is providing, rather than trust it blindly.

I will use 2 AI models. I want one model to handle plant identification, and for this I will use the Pl@ntNet API for plant identification and species classification. The choice for this comes down to believing that the open-source models from this organization being trained for years is going to be better than my attempt to train a CV model like YOLOv11 from Ultralytics myself. The results will be used to query the plant database from Perenual, and any required information about the plant (watering times, health, etc.) will be passed to the AI agent, which I will use the Opus4.5 model from Anthropic for. This agent will use web searching tools to find the missing information on a plant, should the information on the plant be inadequate.

The rest of the software system is structured to use the results from the two models to support the rest of the system. The core of the system will include a system calendar (for tracking watering times of plants), a notification system, a list of owned plants, a list of desired plants, a list of recommended plants, a plant identifying system, a space builder system (where users can build the spaces they own and store specific plants in those spaces), and a CLI for the user to interact with the AI agent. When a user wants to identify a plant, they will use the plant identifier system, which will proceed with the following process:

1. Photo is resized and adjusted appropriately for the Pl@ntNet's requirements
2. The photo is passed to the Pl@ntNet API for plant identification
3. The Pl@ntNet API returns its top 3 guesses for what the plant is, displaying confidence ratings for all of them
4. The user selects the plant it believes most accurately resembles the plant they took a photo of
    a. If none of the guesses look accurate, the user will be invited to share more photos, in which case, afterwards, the process will repeat from step 1
5. The identity of the plant is passed to the Perenual API to determine plant watering information, plant health, etc.
6. The response (information about watering the plant, its current health, etc.) is given back to the system, where it is interpreted by the AI agent and displayed back to the user
    a. If the Perenual API fails (can't find a plant in it's database, failed query, etc.), the system will try to send the query once more, then, upon a second failure, will use the AI agent to search the web for information on the plant
    b. In this case, the AI agent will fill out a template of information that the system will be able to parse and give back to the user

If the user confirms that this is a plant they want to save to their spaces, the AI agent will plot the watering times in the system calendar, and will store the information about the plant in the list of owned plants for it to call back on later, if needed. The agent will also set up notifications for the user (only if it hasn't done so already). Those notifications will be tied directly to the calendar, so the agent won't need to continually update the notifications itself.

From the list of owned plants and prompts from the user, the AI agent will "learn" what kinds of plants the user likes and will keep a list of recommended plants based on the learned patterns. This information (that is, what the agent believes the user likes) will be stored in a separate table in a local database. Furthermore, the user will be able to tell the agent what kinds of spaces they plan to put their plants in. From this, the spaces will be stored in a separate part of the software system, and the agent will be able to use those spaces to refine the recommended plants it gives to the user (for example, if the agent knows the user has a space with lots of direct sunlight, it could recommend plants that benefit from that).

Notifications will be set up to inform users when their plants need to be watered. The system will relate these to the calendar system itself, and so once the AI sets up the watering schedule and frequency of notifications for the user, the system will be able to regulate when to notify the user thereafter.

The app will track how often a user waters their plants so that it can remind users if they haven't watered a plant (it will know this because the app will request that the user indicate which plants they water each time they log into the application. This way, the agent along with the system itself will have the necessary information to provide accurate reminders for the health of various plants.

Lastly, the user will be able to discuss problems with the agent in tandem with the photo identifying tool. The user can prompt the agent for new plant recommendations, for help on caring for a specific plant, and ask the agent to modify the watering schedule it has set up for the user to adapt to a users' schedule. For the last, this can result from the user sharing details about when they would like to water their plants, or from the user sharing an external calendar, to which the agent will interpret and modify the user's application calendar to adapt to these preferences. As well, the agent will be able to accept photos of a plant, should the user have concerns about the health of the plant, and repeat the plant identification process, this time emphasizing the health issues of the plant. The agent will then attempt to educate the user on how to care for the plant, considering its current health and condition.

Major Features (Review before submission for clarity)

F01 - Plant Identifier

The user enters plaintext describing a plant or provides a photo of a plant to the AI agent through the CLI. With plaintext, the input will be processed by the agent. It will propose a few likely scientific names of the plant being described, which are then passed to Perenual's search API. Plants that exist will then be shown to the user for them to confirm the accuracy of, after which the process described above for retrieving plant information will proceed (from step 4 onwards). With a photo, the process described above will proceed from step 1. The AI will then provide the user with information about how to care for the plant, what kind of physical environment it needs to thrive, and will recommend where it should be placed, given that it is aware of the spaces a user has access to (see feature F10). All of this will be displayed in the GUI for the user to see, and to ask more questions about, if needed. If a user indicates that they own the plant, it will be added to their list of owned plants with the appropriate plant information. If information on a plant cannot be successfully retrieved, the agent will attempt to describe a different, but similar, plant that it finds from scraping the web, informing the user of this fact and asking them to retry their input if they desire a different result.

F02 - Plant Watering Schedule Generator

When a user indicates to the AI that they own a certain plant, the agent will plot ideal times to water that plant, using the information it finds about the plant to do this. This information will be plotted in the built-in system calendar for the user. It will attempt to group watering times together with other plants to minimize the complexity of the calendar for a given user. In the case that such a plotting cannot be generated (perhaps because watering times differ too much from other plants or information about watering cannot be found), the AI will attempt to make a schedule with any information it has and will, in the worst case, plot watering times that don't necessarily minimize the complexity of the calendar. If it has no/little/mixed information about how a plant should be cared for, the AI will make this explicit to the user and offer to search for longer for more information, or plot watering times with the information it has.

F03 - Plant Watering Schedule Updater

The user will tell the agent when they have time to water their plants/upload a photo of their schedule to the agent. The AI will then interpret the time constraints given, note any travel time from the user, then update the generated watering schedule in the calendar to accommodate the users' schedule, thus tailoring to the users' availability. In the case that it cannot parse/read/understand the input from the user, the AI will ask for further clarification, either through another photo or through plaintext, and will attempt to rerun the feature workflow.

F04 - Plant Watering Notifications

After a watering schedule for a plant is generated, the software system will align related notifications to the timing outlined in the calendar to remind the user when to water their plants. This will be done deterministically, as the notifications will be tied to a user's system calendar, but the AI will ask (within the discussion it has with a user about the information of a plant) if the user would like to be notified about when the plant should be watered. If yes, the agent will enable this option, but if not, it won't. This option will be toggleable manually as well. Upon interacting with the notification, a list of plants that are expected to be watered, and a list of those that have yet to be watered in the past days will be displayed, with timing for when they should be/should've been watered.

F05 - Plant Health Analyzer

Within the CLI, the user will interact with the agent by describing the condition/sending a photo of one of their plants that they are concerned about, or just want more info about the health of. The agent will determine the appropriate process to handle the given information (the same process described in feature F01), but will instead focus on the health of a plant, highlighting that information for the user. This information will be displayed in the GUI for the user to see, and can be saved within a note for that plant in specific, should the user want to track the health of the plant. In the case that these instructions are inadequate for the user, the AI will reattempt to get more information, and if the user is still dissatisfied, the agent will suggest to the user to input information they know about into the note for that plant.

F06 - Automatic Plant Recommender (Based on what the agent learns about the user)

The AI agent will keep a regularly refreshed list of plants that it believes the user would like to have in their space. It determines its recommendations analyzing the current plants the user has, accounting for the spaces the user has access to (to determine plants best suited for ones with more or less sunlight, for example), accounting for how frequently the user is able to water their plants, and displaying all of this information in a list in the GUI. This list can be sorted by ease of care, name, or recommended space, making it easy for the user to find new plants. The list will also give the user the option to delete or "unrecommend" a plant that they do not like in order to continue learning what the user wants, creating an even more personal experience.

F07 - New Plant Recommender (Based on user input)

The user will notify the agent through the CLI that they are looking for a new plant for their space, and optionally mention the constraints they have for said plant (low ease of care, frequent watering, etc.). The agent will then interpret this information and provide tailored recommendations that relate to the constraints given by the user, saving results the user likes in the list it already has (and ignoring the results the user does not like so as not to populate the list of recommended plants with plants the user actively does not want).

F08 - List of Owned Plants

This is strictly a deterministic system component. Plants that a user owns will be stored in a list of Owned Plants, each with notes about care information for the user to easily access at any time. The user may also enter information about their plants themselves. It could be the case that the agent (in the initial stage of identifying a plant) isn't able to find enough information about the plant to fully populate the notes of some plant. In this case, the agent will leave those bits of the note empty. This list will be available and accessible as part of the GUI of the application.

F09 - List of Desired Plants

To keep track of the plants a user would potentially purchase in the future, or is interested in learning more about, there will also be a list of plants that is recommended to the user. The user will have the option to state where in their spaces they would like the plant to go, and the AI will provide feedback later on about whether or not this is a good choice for the plant desired. This list, as the one above, will be available and accessible as part of the GUI of the application.

F10 - Room Builder

To easily track which plants are in which spaces in a users' home, apartment, or other space, the Room Builder feature can be used. This feature will be available as part of the GUI and will give users the option to build their spaces, adding the plants that are in that space along with it. When a space is selected within the features domain, a list of the plants in the space will be displayed to the user. This list can be updated either by the user or the agent, which, in the agent's case, will be updated if the user indicates to the agent which space a plant they own is located. This may be too much to ask, but I would also like to add to this feature a "zoom in" feature, which would allow users to plot in a specific space where

their plants are exactly, and use that feature to see which plants require water, which are in good health, which have notes about disease, etc. in a very personal way.

F11 - Plant Sharing with Friends

Ever walk into a friend's place and wish you knew what plants they have that make their space so vibrant? With this feature, users will be able to add other friends within the application and view the plants that they own. Other features about a friends' plants, such as plant health, watering needs, etc., will not be available to anyone, aside from the plant owner and their list of allowed viewers.

F12 - Plant Watering Schedule Sharer

Those that the primary plant owner shares the watering schedule with. This is, similar to the feature above, a strictly deterministic part of the system. The user may choose who their friends are and if they want their friends to be able to see their space and plants, or not. Furthermore, with the Plant Watering Schedule Sharer, it becomes easy for users to trust that their friends know exactly how to care for their plants if they, for example, go away for a trip. With this feature, users can share the list of plants, along with their respective water and sunlight needs, to other users, even if those users do not have an account. This ease of access makes it easy to share care instructions from one person to another.

Task 2

See attached .uxf file for the UML Class Diagram of the entire software. I am sorry for the mess, but I couldn't figure out a cleaner way to define all of the relationships.

Choice for Design Patterns

Facade

The GUI (MainWindow) and the CLI (CliConsole) both need to reach almost everything the system does: identification, scheduling, notifications, health analysis, recommendations, spaces and sharing. Each of those lives in a different subsystem, and a client that has to know about all of them would make a client's code tightly coupled to the system, relying on the client to also understand each process. AgentController is the facade, which exposes one simple set of methods such as identifyPlant() and markWatered(), thus coordinating the PlantAgent, ScheduleManager, SpaceManager, ShareService, RecommendationEngine, NotificationService, PlantRepository, and IdentificationResult classes. This fits because the GUI and CLI are required to share some similar functionality, and a single entry point lets them share one implementation. Without it, the GUI and the CLI would each duplicate the classes and their logic.

Adapter

The system depends on several external services that each have their own request format, response shape, authentication and failure modes. For example, Pl@ntNet for identification, Perenual for care data, Claude Opus 4.5 for language tasks, etc. If the core code worked with these directly, their differences would spread everywhere. To fix this, there are classes that act as interfaces (PlantIdentifier, PlantInfoProvider, LLMClient and AvailabilityImporter) that wrap each external service in an adapter, so only one implementation is required. Pl@ntNetAdapter, PerenualAdapter, etc. are the adapters, and the external APIs and file format are what are adapted. This is appropriate because the Perenual-then-web-search fallback only works if both sources look identical to PlantInfoService, for example. Without adapters, a change to one API would require edits across many classes, and since I know choices for tools, for example, may change throughout the implementation, having to edit less core logic when this happens is of major benefit.

Strategy

The system has two places where the better algorithm depends on the situation. For watering schedules (F02), the best algorithm groups waterings with other plants, but that is impossible when the watering intervals are incompatible. In this case, an optimal-time-to-effort schedule is needed instead. For recommendations, learning from the user's history (F06) and satisfying constraints the user states directly (F07) are different procedures that require different algorithms. In the first case, ScheduleManager is the context and uses a SchedulingStrategy, with GroupedWateringStrategy and FallbackWateringStrategy as the concrete strategies. In the second, RecommendationEngine is the context and uses a RecommendationStrategy, with LearnedPreferenceStrategy and UserConstraintStrategy as the concrete strategies. The context calls the interface (plan() or recommend()) and can swap the strategy with setStrategy(). This fits because the choice is made at runtime from the data, and each algorithm can change independently. Without it, ScheduleManager and RecommendationEngine would contain large conditional blocks mixing every algorithm together. Adding a third scheduling or recommendation approach would mean editing and retesting that tangled code instead of adding a new class.

Observer

Notifications are tied directly to the calendar so the agent does not have to keep updating them. To make this process easier, the Observer design pattern allows us to easily update different classes that are dependent on the watering schedule of a plant. Here, ScheduleManager is the subject class. It keeps a list of observers (registerObserver()) and calls notifyObservers(entry) whenever entries are added, rescheduled, completed or undone. CalendarObserver is the observer interface with an update(entry) method, and NotificationService is the concrete observer that adjusts its reminders in response. This is the right pattern because the calendar should not know anything about how notifications work, and the agent should not be responsible for remembering to resync them so as to reduce agent usage. It also leaves room to add another observer later, such as a calendar view, without touching ScheduleManager. Without it, every piece of code that edits the calendar (schedule generation, schedule updates, undo, marking a plant as watered) would need to call the notification code by hand, which could result in missed calls that reduce the effectiveness of the watering schedule and its dependencies.

Factory Method

Two parts of the system need to create objects whose concrete type depends on runtime information. Schedule availability (F03) can arrive as text, as a photo or as an iCal file, and each needs a different parser. The agent also has many tools that must be created and registered. ImporterFactory has a create(inputType) method that returns an AvailabilityImporter, either TextScheduleParser, ImageScheduleParser or ICalAdapter, and ToolFactory.createTool(type) does the same for the Tool implementations, for example. The callers, ScheduleManager and ToolManager, work only with the returned interface and never name a concrete class. This pattern fits well because it keeps the decision of which class to build in one place. Without the pattern, ScheduleManager would contain the construction logic for every parser and would need editing each time a new input type was added, which would be costly and inefficient, not to mention it makes redesign more prone to error.

Composite

The Room Builder (F10) feature lets users model nested spaces, such as a home containing rooms, rooms containing shelves, and shelves containing spots where individual plants sit. A user selecting any part of a room should expect to see all the plants within it. The SpaceComponent class is the abstract component with getPlants() and add(). Room is the composite that holds child components, and Spot is the leaf that holds an actual OwnedPlant. Calling getPlants() on a room recursively collects the plants from its children, while a spot just returns the plants within it. This design pattern is appropriate because without it, we would need code for every kind of spot that could exist in a house, which could get messy very quickly. With this pattern, we can simply add new components to a room and maintain the same code structure to return a list of plants within a room.

Command

The agent can change the user's calendar, and AI-driven changes are sometimes wrong, for example a reschedule that misreads a photographed timetable. Users therefore need a way to reverse an agent action. Each calendar change is wrapped in an object that implements ScheduleCommand, with execute() and undo() methods. CommandHistory is the invoker that runs commands and remembers them, and undoLast() reverses the most recent one. ScheduleManager is the receiver whose entries are modified, and it also acts as the client that creates the commands. This design pattern is appropriate because each command naturally stores what it needs to reverse itself, and the history gives a record of everything the agent did. Without it, undoing an agent reschedule would mean reconstructing the old calendar by hand from whatever state happened to remain, and this is both error-prone and wasteful.

UML Use Case Diagrams

Please see attached .uxf files for the use-case diagrams themselves.

Description of Use Cases

UC01 – Identify Plant (F01)

- Actors: Plant Owner, Pl@ntNet, Perenual, Claude, Web Search
- Goal: Learn what a plant is, how to care for it and where to place it.
- Preconditions: App running; API keys configured; user has a photo or text description.
- Trigger: User submits a photo or text in the GUI chat panel or CLI.
- Main scenario: User submits photo → System resizes it → Pl@ntNet returns top 3 candidates with confidence → User selects the best match → System looks up care info (UC13) → Agent explains care needs and recommends a space using the user's spaces → System asks whether the user owns the plant (UC02).
- Alternatives:
    - Pl@ntNet unreachable: report error, offer retry with web search
    - No candidate matches: invite more photos, repeat process for plant identification
    - Care info unavailable: agent describes a similar plant found online, says so, and asks the user to retry if they want a different result
- Postconditions: User knows the species and care needs of a plant; nothing saved to the owned list unless confirmed.

UC02 – Add Owned Plant (F08)

- Actors: Plant Owner, Claude
- Goal: Store a plant with its care notes.
- Preconditions: A plant was identified (UC01) or the user enters one manually.
- Trigger: User confirms "I own this plant" or chooses "Add plant".
- Main scenario: User confirms ownership and optionally picks a space → System saves OwnedPlant with its care profile → System triggers schedule generation (UC03) → Plant appears in the owned list.
- Alternatives:
    - Missing care data: those note fields are left empty and so the user must type the details if desired and not otherwise available (may ask Claude agent for assistance)
- Postconditions: Plant added to owned plants list; calendar entries and reminders exist.

UC03 – Generate Watering Schedule (F02)

- Actors: Plant Owner, Claude
- Goal: Plot watering times that are easy to follow.
- Preconditions: An owned plant with a care profile exists.
- Trigger: Generated upon user request
- Main scenario: System takes the watering interval → It tries to group watering with existing plants → Entries are saved to the calendar as an undoable command → Observers update notifications → Agent explains the schedule and asks whether the user wants notifications (only on the first time a user uses this function).
- Alternatives:
    - Grouping impossible data or data missing: fall back to a best-effort schedule
    - Little/mixed data: agent says so explicitly and offers a deeper search or to proceed with what it has.
- Postconditions: Calendar entries for watering a plant exist

UC04 – Update Schedule to Availability (F03)

- Actors: Plant Owner, Claude, External Calendar

- Goal: Fit watering times to the owner's real availability.
- Preconditions: At least one schedule exists.
- Trigger: User sends availability as text, a photo of a timetable, or an iCal file.
- Main scenario: System selects a parser for the input type → Parser (using Claude for text/images) extracts availability including travel time → Strategy produces new entries → Reschedule is applied through the command history → Notifications refresh → Agent summarizes the changes.
- Alternatives:
    - Input unreadable: agent asks for another photo or text and reruns
    - User dislikes the result: undo the last change, offer retry
- Postconditions: Calendar matches availability; change is undoable.

UC05 – Receive Reminders and Confirm Watering (F04)

- Actors: Plant Owner, Claude
- Goal: Be reminded to water and record that it was done.
- Preconditions: Calendar entries exist; notifications enabled for watering plants.
- Trigger: Entry becomes due, or the user opens the app/a notification.
- Main scenario: Notification is sent → User opens it and sees plants due and plants overdue with times → User marks plants as watered → Plant returns to OkState; next time to water is scheduled.
- Alternatives:
    - Not confirmed past threshold: plant becomes OverdueState and a missed-watering reminder is sent.
- Postconditions: Watering history updated.

UC06 – Analyze Plant Health (F05)

- Actors: Plant Owner, Pl@ntNet, Claude, Web Search
- Goal: Understand a plant's health problem and what to do.
- Preconditions: Plant is in the owned list.
- Trigger: User describes a problem or sends a photo of a plant to Claude agent.
- Main scenario: Agent confirms the species (unless function is called within a plant's notes) → It loads previous photos/records and care info (UC13) → Claude assesses the plant and compares with earlier photos (improving/worsening) → New photo and health record are stored → Instructions are shown, letting user save them to the plant's note.
- Alternatives:
    - User unsatisfied: agent searches the web for more details
    - Still unsatisfied: agent suggests the user add their own notes (LOL)
- Postconditions: Health history extended.

UC07 – View Automatic Recommendations (F06)

- Actors: Plant Owner, Claude
- Goal: Give user the option to browse plants the agent thinks the user will like.
- Preconditions: Some owned plants or learned preferences exist.
- Trigger: Timer or change to the owned list refreshes the list; user opens the list.

- Main scenario: Agent proposes species from owned plants, spaces, watering ability and preferences → Each is validated against Perenual → User sorts by ease of care, name or space → User may "unrecommend" a plant.
- Alternatives:
    - No data yet: list is empty with a prompt to add plants
    - Unrecommend: dislike stored in agent memory, plant removed from recommended list
- Postconditions: List refreshed; preferences updated.

UC08 – Request Recommendations (F07)

- Actors: Plant Owner, Claude
- Goal: Get recommendations for stated constraints.
- Preconditions: None.
- Trigger: User asks in the CLI/chat for a plant with constraints.
- Main scenario: Agent interprets constraints → Candidates are generated and filtered against care profiles → User sees results → Liked results join the recommendation list.
- Alternatives:
    - User dislikes recommendation: Disliked results go to memory and are not shown again
    - No match: agent suggests relaxing constraints.
- Postconditions: List and preferences updated.

UC09 – View Desired Plants (F09)

- Actors: Plant Owner, Claude
- Goal: Track plants the user might buy, with an intended space.
- Preconditions: App is running as expected.
- Trigger: User adds/edits a desired plant.
- Main scenario: User opens list of recommended plants → list of recommended plants is displayed to user
- Alternatives:
    - No care info: agent says it cannot judge and offers a search.
- Postconditions: Desired list persisted.

UC10 – Manage Spaces (F10)

- Actors: Plant Owner, Claude
- Goal: Model rooms and spots and place plants in them.
- Preconditions: App is running as expected.
- Trigger: User opens the Room Builder, or tells the agent where a plant is.
- Main scenario: User creates a room (with light level) and spots → User assigns plants → Selecting a space lists its plants (recursively).
- Alternatives:
    - Agent assigns the plant when the user states its location
    - Plant already placed: it is moved.
- Postconditions: Space tree and assignments saved.

UC11 – View Friends' Plants (F11)

- Actors: Plant Owner, Friend
- Goal: See which plants a friend owns.
- Preconditions: Users are friends and the owner allows viewing.
- Trigger: User adds a friend or opens a friend's list.
- Main scenario: User adds a friend → Friend (an allowed viewer) opens the owner's list → System shows plant names only.
- Alternatives:
    - Access denied: User cannot view plants of their friend
- Postconditions: Friendship stored.

UC12 – Share Watering Schedule (F12)

- Actors: Plant Owner, Guest
- Goal: Let someone without an account care for the plants.
- Preconditions: Owner has plants.
- Trigger: Owner selects plants and creates a share link.
- Main scenario: Owner selects plants → System creates a token link → Guest opens it → System shows plants, watering schedule and sunlight needs only.
- Alternatives:
    - Invalid token: error page.
- Postconditions: Link stored.

UC13 – Look Up Care Info (included by UC01 and UC06)

- Actors: Perenual, Web Search, Claude
- Goal: Obtain a PlantCareProfile for a species.
- Preconditions: Species name known.
- Trigger: Included by UC01/UC06.
- Main scenario: Query Perenual → Return the profile.
- Alternatives:
    - Perenual fails: retry once
    - Second failure: agent searches the web and fills a care template, marked "from web, lower reliability".
    - Still nothing: report it and offer a similar plant.
- Postconditions: Profile available (and cached).

Sequence Diagrams

Please see attached .uxf files for sequence diagrams.

Task 3

Feature	Description	Type	Use Case	Classes	Key Methods	Sequence Diagram	Design Pattern(s)
F01	Plant identifier (photo or text) with care and placement info	Hybrid	UC01, UC13	MainWindow, CliConsole, AgentController, PlantAgent, AgentPlanner, ToolManager, IdentifyPlantTool, PlantInfoTool, ImageProcessor, PlantNetAdapter, PlantInfoService, PerenualAdapter, WebSearchAdapter, ClaudeAdapter, SpaceManager	identifyPlant(), identifyByText(), confirmCandidate(), run(), createPlan(), executeTool(), identify(), lookup(), searchByName(), explain()	SD01, SD02	Facade, Adapter, Factory Method
F02	Watering schedule generator that groups waterings	Hybrid	UC03 (via UC02)	AgentController, PlantAgent, CalendarTool, ScheduleManager, SchedulingStrategy, GroupedWateringStrategy, FallbackWateringStrategy, CommandHistory, AddEntriesCommand, CalendarEntry	addOwnedPlant(), generate(), plan(), setStrategy(), run(), notifyObservers()	SD03	Strategy, Observer
F03	Schedule updater from text, photo or iCal availability	Hybrid	UC04	AgentController, PlantAgent, CalendarTool, ScheduleManager, ImporterFactory, AvailabilityImporter, TextScheduleParser, ImageScheduleParser, ICalAdapter, Availability, RescheduleCommand, CommandHistory, ClaudeAdapter	updateSchedule(), applyAvailability(), create(), parse(), plan(), undoScheduleChange(), undoLast()	SD04	Factory Method, Adapter, Strategy
F04	Watering notifications tied to the calendar	Hybrid	UC05	ScheduleManager, CalendarObserver, NotificationService, Notification, WateringOverview, OwnedPlant, WateringState, OkState, DueState, OverdueState, AgentController	update(), checkDueEntries(), send(), getOverdue(), getUpcoming(), setEnabled(), markWatered(), onTick(), onWatered(), completeEntry()	SD05 (opt-in in SD03)	Observer, State

F05	Plant health analyzer with photo history	AI	UC06 (includes UC13)	AgentController, PlantAgent, HealthAnalysisTool, WebSearchTool, PlantNetAdapter, PlantInfoService, ClaudeAdapter, HealthRecord, PlantPhoto, PlantNote, PlantRepository	analyzeHealth(), execute(), compareTo(), savePhoto(), saveHealthRecord(), saveNote(), search()	SD06	Facade, Adapter
F06	Automatic recommender from learned preferences	Hybrid	UC07	RecommendationEngine, LearnedPreferenceStrategy, RecommendationContext, RecommendedPlant, AgentMemory, UserPreferenceModel, PlantInfoService, ClaudeAdapter, AgentController	refresh(), recommend(), getPreferences(), sort(), unrecommend(), savePreference(), sortRecommendations()	SD07	Strategy, Facade
F07	Recommender from user constraints	AI	UC08	AgentController, PlantAgent, RecommendTool, RecommendationEngine, UserConstraintStrategy, AgentMemory, ClaudeAdapter	getRecommendations(), setStrategy(), recommendFor(), recommend(), accept(), unrecommend()	SD07	Strategy, Facade
F08	Owned plants list	Deterministic	UC02	AgentController, PlantRepository, OwnedPlant, PlantCareProfile, PlantNote, DatabaseManager	addOwnedPlant(), save(), findAll(), findById(), saveNote()	SD03	Facade (Repository-style persistence)
F09	Desired plants list	Hybrid	UC09	AgentController, PlantRepository, DesiredPlant, SpaceManager, PlantAgent, ClaudeAdapter	addDesiredPlant(), saveDesired(), findDesiredById(), checkDesiredPlantFit(), getSpace()	SD08	Facade
F10	Room builder for user spaces	Hybrid	UC10	SpaceManager, SpaceComponent, Room, Spot, SpaceRepository, SpaceTool, ToolManager, PlantAgent, AgentController	createSpace(), assignPlant(), getPlants(), listSpaces(), add(), assign()	SD08	Composite, Facade
F11	Plant sharing with friends	Deterministic	UC11	ShareService, User, UserRepository, PlantRepository, PlantSummary, AgentController	addFriend(), viewFriendPlants(), findById()	SD09	Facade

F12	Watering schedule sharer via link, no account	Deterministic	UC12	ShareService, ShareLink, SharedScheduleView, OwnedPlant, UserRepository, PlantRepository, AgentController	createShareLink(), openShareLink(), saveShareLink(), findShareLink(), findByIds()	SD09	Facade

Task 4 - How Each Feature is Realized

F01 – Plant Identifier
Related Use Cases: UC01, UC13
Related Sequence Diagrams: SD01, SD02

Classes Involved:

- MainWindow, CliConsole: Collect the photo or text description from the user and display the results
- AgentController: Facade and single entry point for both interfaces
- SpaceManager: Supplies the user's spaces so placement advice can be given
- PlantAgent: Runs the request and writes the care and placement explanation
- AgentPlanner: Chooses the steps for the request (photo path or text path)
- ToolManager: Selects and executes the appropriate tool
- IdentifyPlantTool: Runs photo identification
- PlantInfoTool: Runs the care-info lookup and species search
- ImageProcessor: Resizes photos to Pl@ntNet's requirements
- PlantNetAdapter: Calls Pl@ntNet and returns the top 3 candidates
- PlantInfoService: Finds care data, with retry and fallback logic
- PerenualAdapter: Queries Perenual for care profiles and species names
- WebSearchAdapter: Fallback source when Perenual fails twice
- ClaudeAdapter: Proposes scientific names for text input, fills the care template, and writes the explanation

Important Methods:

- MainWindow.submitRequest(text, photo)
- AgentController.identifyPlant(photo)
- AgentController.identifyByText(description)
- AgentController.confirmCandidate(candidate)
- SpaceManager.listSpaces()
- PlantAgent.run(request)
- AgentPlanner.createPlan(request)
- ToolManager.executeTool(name, args)
- IdentifyPlantTool.execute(args)
- PlantInfoTool.execute(args)
- ImageProcessor.resize(photo)
- PlantNetAdapter.identify(photo)
- PlantInfoService.lookup(species)
- PlantInfoService.searchByName(name)
- PerenualAdapter.getCareProfile(species)
- WebSearchAdapter.getCareProfile(species)
- ClaudeAdapter.generateStructured(prompt, template)

- PlantAgent.explain(profile, spaces)
- ClaudeAdapter.generate(prompt, context)

Execution: For a photo, MainWindow.submitRequest() passes the input to AgentController.identifyPlant(), which calls PlantAgent.run(). AgentPlanner.createPlan() decides the steps, and ToolManager.executeTool() runs IdentifyPlantTool.execute(). The tool calls ImageProcessor.resize() and then PlantNetAdapter.identify(), which returns three candidates with confidence. The user picks one and AgentController.confirmCandidate() starts the care lookup: PlantInfoService.lookup() calls PerenualAdapter.getCareProfile(), retries once, then falls back to WebSearchAdapter.getCareProfile(), where ClaudeAdapter.generateStructured() fills the care template. AgentController gets the user's spaces from SpaceManager.listSpaces(), and PlantAgent.explain() combines the profile with them through ClaudeAdapter.generate() to explain how to care for the plant and where to place it. For text, AgentController.identifyByText() leads PlantAgent.run() to ask ClaudeAdapter.generateStructured() for likely scientific names, and PlantInfoService.searchByName() checks which exist in Perenual; the flow then joins the photo path at AgentController.confirmCandidate(). If no care information can be found, PlantAgent describes a similar plant, says so, and asks the user to retry.

F02 – Watering Schedule Generator

Related Use Cases: UC03 (included by UC02)

Related Sequence Diagrams: SD03

Classes Involved:

- AgentController: Receives the ownership confirmation and starts schedule generation
- PlantRepository: Saves the new owned plant and supplies existing plants for grouping
- PlantAgent: Runs the schedule-generation request and explains the result
- ToolManager: Executes the calendar tool
- CalendarTool: Bridges the agent to the scheduling subsystem
- ScheduleManager: Owns the calendar entries and chooses the strategy
- GroupedWateringStrategy: Aligns watering with existing plants
- FallbackWateringStrategy: Produces a best-effort schedule when grouping fails
- CommandHistory, AddEntriesCommand: Save the entries as an undoable command
- CalendarEntry: One planned watering
- NotificationService: Observer updated when entries are added
- ClaudeAdapter: Writes the schedule explanation and data-quality notice

Important Methods:

- AgentController.addOwnedPlant(profile, spaceId)
- PlantRepository.save(plant)
- PlantAgent.run(request)
- ToolManager.executeTool(name, args)
- CalendarTool.execute(args)
- ScheduleManager.generate(plant)
- PlantRepository.findAll()
- GroupedWateringStrategy.plan(plants, availability)
- ScheduleManager.setStrategy(s)
- FallbackWateringStrategy.plan(plants, availability)
- CommandHistory.run(cmd)
- AddEntriesCommand.execute()

- ScheduleManager.notifyObservers(entry)
- NotificationService.update(entry)
- ClaudeAdapter.generate(prompt, context)

Execution: After the user confirms ownership, AgentController.addOwnedPlant() saves the plant with PlantRepository.save() and asks PlantAgent.run() to generate a schedule. ToolManager.executeTool() calls CalendarTool.execute(), which calls ScheduleManager.generate(). The manager loads existing plants with PlantRepository.findAll() and tries GroupedWateringStrategy.plan() to align the new plant with them. If grouping is impossible or the data is missing, ScheduleManager.setStrategy() switches to FallbackWateringStrategy and FallbackWateringStrategy.plan() produces a best-effort schedule. The entries are stored through CommandHistory.run() and AddEntriesCommand.execute() so they can be undone, and ScheduleManager.notifyObservers() lets NotificationService.update() refresh reminders. PlantAgent then uses ClaudeAdapter.generate() to explain the schedule, state clearly when the data was thin, and offer a deeper search.

F03 – Watering Schedule Updater
Related Use Cases: UC04
Related Sequence Diagrams: SD04

Classes Involved:

- AgentController: Receives the availability input and the undo request
- PlantAgent: Runs the update request and summarizes the result or asks for clarification
- ToolManager: Executes the calendar tool
- CalendarTool: Bridges the agent to the scheduling subsystem
- ScheduleManager: Applies availability and rebuilds the schedule
- ImporterFactory: Picks the parser for the input type
- TextScheduleParser, ImageScheduleParser, ICalAdapter: Turn text, photos or iCal files into availability
- Availability: Time slots and travel time
- ClaudeAdapter: Extracts availability and travel time from text and images
- SchedulingStrategy (GroupedWateringStrategy, FallbackWateringStrategy): Produces the new entries
- CommandHistory, RescheduleCommand: Apply and undo the change
- NotificationService: Observer updated when entries change

Important Methods:

- AgentController.updateSchedule(input)
- PlantAgent.run(request)
- ToolManager.executeTool(name, args)
- CalendarTool.execute(args)
- ScheduleManager.applyAvailability(input)
- ImporterFactory.create(inputType)
- AvailabilityImporter.parse(input) (implemented by TextScheduleParser, ImageScheduleParser and ICalAdapter)
- ClaudeAdapter.generateStructured(prompt, template)
- SchedulingStrategy.plan(plants, availability)
- CommandHistory.run(cmd)
- RescheduleCommand.execute()
- ScheduleManager.notifyObservers(entry)
- NotificationService.update(entry)
- AgentController.undoScheduleChange()

- ScheduleManager.undoLast()
- CommandHistory.undoLast()
- RescheduleCommand.undo()

Execution: AgentController.updateSchedule() passes the input to PlantAgent.run(), which calls CalendarTool.execute() through ToolManager.executeTool(). CalendarTool calls ScheduleManager.applyAvailability(), which asks ImporterFactory.create() for the right parser and calls AvailabilityImporter.parse(). The text and image parsers use ClaudeAdapter.generateStructured() to extract availability including travel time, while ICalAdapter parses deterministically. SchedulingStrategy.plan() produces the new entries, CommandHistory.run() applies them through RescheduleCommand.execute(), and ScheduleManager.notifyObservers() updates reminders. If the input cannot be parsed, a ParseError returns to PlantAgent, which asks for another photo or text and reruns the flow. If the user dislikes the result, AgentController.undoScheduleChange() calls ScheduleManager.undoLast(), which calls CommandHistory.undoLast() and RescheduleCommand.undo().

F04 – Watering Notifications
Related Use Cases: UC05
Related Sequence Diagrams: SD05 (the opt-in step appears in SD03)

Classes Involved:

- MainWindow: Shows notifications and collects "mark as watered" and toggle actions
- AgentController: Routes watering and notification requests
- ScheduleManager: Subject of the Observer pattern; provides due entries and completes entries
- CalendarObserver: Observer interface
- NotificationService: Concrete observer that sends and tracks reminders
- Notification, WateringOverview: Data shown to the user
- OwnedPlant: Context of the State pattern
- WateringState, OkState, DueState, OverdueState: Watering-status behaviour
- PlantRepository: Loads and saves plants

Important Methods:

- ScheduleManager.notifyObservers(entry)
- NotificationService.update(entry)
- NotificationService.checkDueEntries()
- ScheduleManager.getDueEntries(time)
- PlantRepository.findById(id)
- OwnedPlant.checkDue()
- WateringState.onTick(plant)
- OwnedPlant.setState(state)
- NotificationService.send(notification)
- AgentController.getWateringOverview()
- NotificationService.getUpcoming()
- NotificationService.getOverdue()
- MainWindow.markWatered(plantId)
- AgentController.markWatered(plantId)
- OwnedPlant.markWatered()
- WateringState.onWatered(plant)
- PlantRepository.save(plant)
- ScheduleManager.completeEntry(plantId)

- MainWindow.toggleNotifications(plantId, enabled)
- AgentController.setNotifications(plantId, enabled)
- NotificationService.setEnabled(plantId, enabled)

Execution: Whenever calendar entries change, ScheduleManager.notifyObservers() calls NotificationService.update(), so reminders follow the calendar without the agent managing them. On a timer and at app start, NotificationService.checkDueEntries() calls ScheduleManager.getDueEntries(), loads each plant with PlantRepository.findById(), and calls OwnedPlant.checkDue(). That calls WateringState.onTick(), which moves the plant to DueState or, past its threshold, OverdueState through OwnedPlant.setState(); NotificationService.send() then delivers the reminder if notifications are enabled. When the user opens a notification, AgentController.getWateringOverview() combines NotificationService.getUpcoming() and NotificationService.getOverdue() into a WateringOverview. When the user marks a plant as watered, AgentController.markWatered() calls OwnedPlant.markWatered(), whose WateringState.onWatered() returns the plant to OkState; the controller then saves it with PlantRepository.save() and calls ScheduleManager.completeEntry(), which notifies observers about the next entry. Notifications are toggled per plant with AgentController.setNotifications(), which calls NotificationService.setEnabled().

F05 – Plant Health Analyzer

Related Use Cases: UC06 (includes UC13)

Related Sequence Diagrams: SD06

Classes Involved:

- MainWindow, CliConsole: Collect the description or photo and display the result
- AgentController: Entry point for the health request and for saving notes
- PlantAgent: Runs the request and writes the educational explanation
- ToolManager: Executes the health and web-search tools
- HealthAnalysisTool: Orchestrates the analysis
- PlantRepository: Loads plant history; stores photos, records and notes
- PlantNetAdapter: Confirms the species from the photo
- PlantInfoService: Provides the care profile
- ClaudeAdapter: Assesses the plant against its history
- HealthRecord, PlantPhoto, PlantNote: Health history and notes
- WebSearchTool, WebSearchAdapter: Find extra information when the user is unsatisfied

Important Methods:

- AgentController.analyzeHealth(plantId, photo, description)
- PlantAgent.run(request)
- ToolManager.executeTool(name, args)
- HealthAnalysisTool.execute(args)
- PlantRepository.findById(id)
- PlantNetAdapter.identify(photo)
- PlantInfoService.lookup(species)
- ClaudeAdapter.generate(prompt, context)
- HealthRecord.compareTo(other)
- PlantRepository.savePhoto(plantId, photo)
- PlantRepository.saveHealthRecord(plantId, record)
- AgentController.saveNote(plantId, text)

- PlantRepository.saveNote(plantId, note)
- WebSearchTool.execute(args)
- WebSearchAdapter.search(query)

Execution: AgentController.analyzeHealth() sends the request to PlantAgent.run(), which calls HealthAnalysisTool.execute() through ToolManager.executeTool(). The tool loads the plant and its earlier photos and records with PlantRepository.findById(), confirms the species with PlantNetAdapter.identify() when a photo was supplied, and gets the care profile from PlantInfoService.lookup(). ClaudeAdapter.generate() assesses the plant against the earlier photos, and HealthRecord.compareTo() reports whether it is improving or worsening. The new photo and record are stored with PlantRepository.savePhoto() and PlantRepository.saveHealthRecord(), and PlantAgent explains the care steps and the reasoning. The user can save the result with AgentController.saveNote(), which calls PlantRepository.saveNote(). If the user is unsatisfied, PlantAgent runs WebSearchTool.execute(), which calls WebSearchAdapter.search(); if the user is still unsatisfied, the agent suggests adding their own notes.

F06 – Automatic Plant Recommender
Related Use Cases: UC07
Related Sequence Diagrams: SD07

Classes Involved:

- AgentController: Triggers refresh and handles sorting and "unrecommend" requests
- RecommendationEngine: Context of the Strategy pattern; holds the recommended list
- LearnedPreferenceStrategy: Recommends from owned plants, spaces and learned preferences
- RecommendationContext: Inputs passed to the strategy
- RecommendedPlant: One recommendation with reason and score
- AgentMemory, UserPreferenceModel: Stored likes and dislikes
- ClaudeAdapter: Proposes candidate species
- PlantInfoService: Validates each candidate against care data

Important Methods:

- RecommendationEngine.refresh()
- RecommendationStrategy.recommend(context) (implemented by LearnedPreferenceStrategy)
- AgentMemory.getPreferences()
- ClaudeAdapter.generateStructured(prompt, template)
- PlantInfoService.lookup(species)
- AgentController.sortRecommendations(by)
- RecommendationEngine.sort(by)
- AgentController.unrecommend(id)
- RecommendationEngine.unrecommend(id)
- AgentMemory.savePreference(key, value)

Execution: RecommendationEngine.refresh() runs on a timer or when the owned list changes. It calls LearnedPreferenceStrategy.recommend() with a RecommendationContext built from owned plants, spaces, watering ability and the preferences returned by AgentMemory.getPreferences(). The strategy asks ClaudeAdapter.generateStructured() for candidate species and validates each through PlantInfoService.lookup(). The user can sort the list with AgentController.sortRecommendations(), which calls RecommendationEngine.sort(). When the user rejects an item, AgentController.unrecommend() calls RecommendationEngine.unrecommend(), which removes it and calls AgentMemory.savePreference() to record the dislike.

F07 – New Plant Recommender (User Input)

Related Use Cases: UC08

Related Sequence Diagrams: SD07

Classes Involved:

- MainWindow, CliConsole: Collect the user's constraints
- AgentController: Entry point and acceptance/rejection handling
- PlantAgent: Interprets the request
- ToolManager: Executes the recommendation tool
- RecommendTool: Bridges the agent to the recommendation subsystem
- RecommendationEngine: Switches strategy and returns the filtered list
- UserConstraintStrategy: Recommends from the stated constraints
- ClaudeAdapter: Proposes candidate species
- PlantInfoService: Provides care profiles for filtering
- AgentMemory: Stores dislikes

Important Methods:

- MainWindow.submitRequest(text, photo)
- AgentController.getRecommendations(constraints)
- PlantAgent.run(request)
- ToolManager.executeTool(name, args)
- RecommendTool.execute(args)
- RecommendationEngine.setStrategy(s)
- RecommendationEngine.recommendFor(constraints)
- RecommendationStrategy.recommend(context) (implemented by UserConstraintStrategy)
- ClaudeAdapter.generateStructured(prompt, template)
- PlantInfoService.lookup(species)
- AgentController.acceptRecommendation(id)
- RecommendationEngine.accept(id)
- AgentController.unrecommend(id)
- RecommendationEngine.unrecommend(id)
- AgentMemory.savePreference(key, value)

Execution: The user states their constraints and AgentController.getRecommendations() calls PlantAgent.run(). The agent uses ToolManager.executeTool() to run RecommendTool.execute(), which calls RecommendationEngine.setStrategy() with UserConstraintStrategy and then RecommendationEngine.recommendFor(). The strategy asks ClaudeAdapter.generateStructured() for candidate species and filters them against care profiles from PlantInfoService.lookup(). Results the user keeps are added to the same list as F06 through AgentController.acceptRecommendation() and RecommendationEngine.accept(); rejected ones are recorded as dislikes through AgentController.unrecommend() and AgentMemory.savePreference().

F08 – List of Owned Plants

Related Use Cases: UC02

Related Sequence Diagrams: SD03

Classes Involved:

- MainWindow: Displays the owned-plants list
- AgentController: Handles ownership confirmation and manual entry

- PlantRepository: Saves and loads owned plants and their notes
- OwnedPlant: The stored plant, with its state and care data
- PlantCareProfile: Care information attached to the plant
- PlantNote: User and agent notes (left empty if care data is missing)
- DatabaseManager: Executes the SQL for the repository

Important Methods:

- AgentController.addOwnedPlant(profile, spaceId)
- PlantRepository.save(plant)
- PlantRepository.findAll()
- PlantRepository.findById(id)
- PlantRepository.saveNote(plantId, note)
- DatabaseManager.execute(sql)
- DatabaseManager.query(sql)

Execution: This feature is fully deterministic. After the user confirms ownership, AgentController.addOwnedPlant() builds the OwnedPlant and calls PlantRepository.save(), which writes it through DatabaseManager.execute(). Missing care data leaves the corresponding note fields empty. The GUI list loads plants with PlantRepository.findAll(), and a single plant with PlantRepository.findById(). Notes are stored with PlantRepository.saveNote(). Manual entry uses the same path as identified plants.

F09 – List of Desired Plants

Related Use Cases: UC09

Related Sequence Diagrams: SD08

Classes Involved:

- MainWindow: Displays the desired list and lets the user choose an intended space
- AgentController: Handles adding plants and the fit check
- PlantRepository: Saves and loads desired plants
- DesiredPlant: A plant of interest, with an optional intended space
- SpaceManager, SpaceComponent: Provide the intended space and its light level
- PlantAgent: Runs the fit check
- ClaudeAdapter: Compares the plant's needs to the space

Important Methods:

- AgentController.addDesiredPlant(desired)
- PlantRepository.saveDesired(plant)
- AgentController.checkDesiredPlantFit(desiredId)
- PlantRepository.findDesiredById(id)
- SpaceManager.getSpace(spaceId)
- PlantAgent.run(request)
- ClaudeAdapter.generate(prompt, context)

Execution: Saving is deterministic: AgentController.addDesiredPlant() calls PlantRepository.saveDesired() with an optional intended space. Later, AgentController.checkDesiredPlantFit() loads the plant with PlantRepository.findDesiredById() and the space with SpaceManager.getSpace(), then calls PlantAgent.run(). The agent uses ClaudeAdapter.generate() to compare the plant's needs with the space's light and reports whether it is a good fit; if care data is unavailable, it says so and offers to search.

F10 – Room Builder

Related Use Cases: UC10

Related Sequence Diagrams: SD08

Classes Involved:

- MainWindow: Room Builder view where users create spaces and place plants
- AgentController: Entry point for space and placement actions
- SpaceManager: Creates spaces and assigns plants
- SpaceComponent: Composite component with Room (composite) and Spot (leaf)
- SpaceRepository: Saves and loads the space tree
- PlantRepository: Loads and saves the plant being placed
- PlantAgent, ToolManager, SpaceTool: Let the agent place a plant when the user states its location

Important Methods:

- AgentController.createSpace(name, light, parentId)
- SpaceManager.createSpace(name, light, parentId)
- SpaceComponent.add(c)
- SpaceRepository.save(space)
- AgentController.assignPlant(plantId, spaceId)
- SpaceManager.assignPlant(plantId, spaceId)
- PlantRepository.findById(id)
- PlantRepository.save(plant)
- AgentController.getPlantsInSpace(spaceId)
- SpaceManager.getPlants(spaceId)
- SpaceComponent.getPlants()
- AgentController.listSpaces()
- SpaceManager.listSpaces()
- SpaceTool.execute(args)

Execution: AgentController.createSpace() calls SpaceManager.createSpace(), which builds a Room or Spot, attaches it with SpaceComponent.add(), and stores it with SpaceRepository.save(). AgentController.assignPlant() calls SpaceManager.assignPlant(), which loads the plant with PlantRepository.findById(), attaches it to a Spot, and saves both with SpaceRepository.save() and PlantRepository.save(). When the user selects a space, AgentController.getPlantsInSpace() calls SpaceManager.getPlants(), which calls SpaceComponent.getPlants() and recurses through the composite so any level lists its plants. If the user tells the agent where a plant is, PlantAgent runs SpaceTool.execute() through ToolManager.executeTool(), which calls SpaceManager.assignPlant(). The zoom-in stretch goal uses Spots with x/y coordinates, rendered with each plant's watering state and notes.

F11 – Plant Sharing with Friends

Related Use Cases: UC11

Related Sequence Diagrams: SD09

Classes Involved:

- MainWindow: Friend list and friend-plant views
- AgentController: Entry point for friend actions
- ShareService: Manages friendships and enforces who can view what
- User: Holds friend and allowed-viewer lists

- UserRepository: Saves and loads users and friendships
- PlantRepository: Supplies the owner's plants
- PlantSummary: Names-only view of a plant returned to friends

Important Methods:

- AgentController.addFriend(userId)
- ShareService.addFriend(userId)
- UserRepository.save(user)
- AgentController.viewFriendPlants(friendId)
- ShareService.viewFriendPlants(friendId)
- UserRepository.findById(id)
- PlantRepository.findAll()

Execution: AgentController.addFriend() calls ShareService.addFriend(), which links the two users and stores the change with UserRepository.save(). When a friend opens the owner's list, AgentController.viewFriendPlants() calls ShareService.viewFriendPlants(). The service checks the viewer's permission through UserRepository.findById(); if allowed, it loads the plants with PlantRepository.findAll() and returns only PlantSummary names, so health and watering data are never exposed. Otherwise it returns an access-denied result.

F12 – Watering Schedule Sharer

Related Use Cases: UC12

Related Sequence Diagrams: SD09

Classes Involved:

- MainWindow: Lets the owner pick plants and shows the shared view to a guest
- AgentController: Entry point for creating and opening links
- ShareService: Creates links and builds the shared view
- ShareLink: Token plus the chosen plant IDs
- SharedScheduleView: Plants, watering needs and sunlight needs only
- UserRepository: Stores and finds share links
- PlantRepository: Supplies the selected plants

Important Methods:

- AgentController.createShareLink(plantIds)
- ShareService.createShareLink(plantIds)
- UserRepository.saveShareLink(link)
- AgentController.openShareLink(token)
- ShareService.openShareLink(token)
- UserRepository.findShareLink(token)
- PlantRepository.findByIds(ids)

Execution: AgentController.createShareLink() calls ShareService.createShareLink(), which creates a ShareLink with a token and the chosen plant IDs and stores it with UserRepository.saveShareLink(). A guest opens the link without an account: AgentController.openShareLink() calls ShareService.openShareLink(), which finds the link with UserRepository.findShareLink(), loads the plants with PlantRepository.findByIds(), and returns a SharedScheduleView containing only plant names, watering needs and sunlight needs. An invalid token returns an error.